

BIOLOGÍA DE SISTEMAS: DE GENES A SOCIEDADES

Capítulo 6

Simular el azar: el algoritmo de Gillespie

EN ESTE CAPÍTULO

De Doob a Gillespie: los orígenes del método · Por qué el paso de tiempo fijo es un mal camino · Muestrear tiempos exponenciales y una primera versión del algoritmo (un número aleatorio por reacción) · Que el primero de varios relojes exponenciales es de nuevo exponencial, y el método directo (un número para el tiempo, otro para el evento) · El caso de muchas reacciones y el costo de generar números aleatorios · El algoritmo en acción.

© 2026 Juan M. Pedraza. Todos los derechos reservados.

0 Del promedio a la trayectoria

En el capítulo 4 vimos que la expresión génica es **intrínsecamente estocástica** y que su descripción correcta es una **distribución de probabilidad** que obedece la ecuación maestra. En el capítulo 5 resolvimos esa ecuación *a mano* para unos pocos casos sencillos. Pero la mayoría de los circuitos reales son demasiado complicados para una solución analítica. Cuando eso pasa, **simulamos**.

Simular significa dejar que la computadora **juegue los dados** por nosotros: generar, una por una, las trayectorias que seguiría una célula individual —las líneas delgadas de la figura 2C del capítulo 4—. Con muchas de esas trayectorias reconstruimos la distribución completa. La herramienta estándar para hacerlo, de forma **exacta**, es el **algoritmo de Gillespie**.

Lo construiremos desde cero: primero su historia, luego por qué el enfoque ingenuo (avanzar en pasos de tiempo fijos) falla, y a partir de un solo hecho del capítulo 4 —que los tiempos entre eventos son exponenciales— llegaremos al algoritmo, mejorándolo paso a paso hasta su forma eficiente.

ANTES DE EMPEZAR

Solo hace falta un resultado del capítulo 4 (el tiempo hasta el siguiente evento de una reacción de tasa k es exponencial, con media $1/k$) y un poco de probabilidad. Escribiremos el método como *pseudocódigo*, no como un programa concreto. Notación: n = número de moléculas, a_i = tasa (propensidad) de la reacción i , $a_0 = \sum a_i$, y u_1, u_2 = números aleatorios uniformes en $(0, 1)$.

1 De Doob a Gillespie

La matemática que hace posible simular exactamente estos procesos es anterior a la biología de sistemas. En los años cuarenta, el matemático **Joseph Doob** formalizó los **procesos de Markov de salto** en tiempo continuo y demostró los dos hechos que necesitaremos: (i) el tiempo que un sistema **espera** en un estado antes de saltar es **exponencial**; y (ii) cuando salta, va a cada estado posible con probabilidad **proporcional a la tasa** de esa transición.

Décadas más tarde, **Daniel Gillespie** (1976, 1977), trabajando en cinética química, convirtió esos hechos en una **receta práctica** para simular moléculas que reaccionan: el *algoritmo de simulación estocástica* (SSA), hoy conocido universalmente como el **algoritmo de Gillespie** (a veces, Doob–Gillespie).

Lo esencial: el algoritmo **no aproxima**. Cada corrida produce una muestra de la *misma* distribución que describe la ecuación maestra del capítulo 4. Una corrida es una posible trayectoria de una célula; muchas corridas, la distribución completa.

2 Por qué no sirve el paso de tiempo fijo

El enfoque ingenuo sería avanzar el tiempo en pasos pequeños Δt y, en cada paso, hacer ocurrir cada reacción con probabilidad **(tasa) $\cdot \Delta t$** (como en la nota al pie de §4.2 del capítulo 4). Funciona, pero mal (Figura 1):

- Si Δt es **grande**, se rompe la suposición de “a lo sumo un evento por paso”: pueden ocurrir dos eventos en un paso, y $(\text{tasa}) \cdot \Delta t$ puede incluso pasar de 1. El resultado es **inexacto**.
- Si Δt es **pequeño**, en casi todos los pasos **no pasa nada**: se gasta enorme cómputo en no hacer nada.
- Elegir Δt es siempre un compromiso entre exactitud y velocidad — y de cualquier modo es **aproximado**.

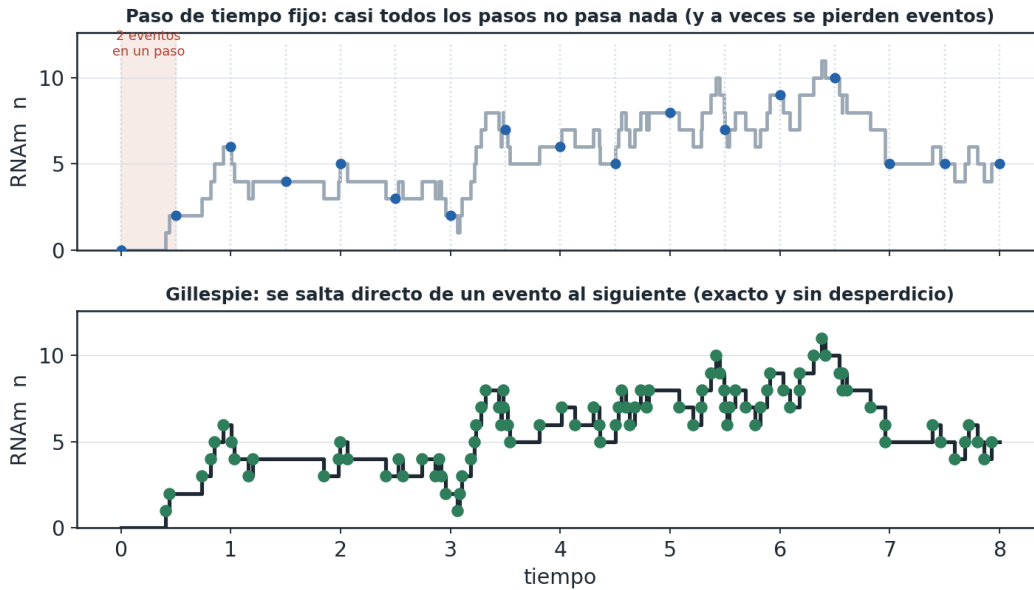


Figura 1. Arriba, el paso de tiempo fijo: la mayoría de los pasos (puntos) caen donde no ocurre nada, y con Δt grande dos eventos caben en un paso (se pierden). Abajo, Gillespie: se salta exactamente de un evento al siguiente.

La idea de Gillespie es la contraria: no marcar el reloj tic a tic, sino **saltar directamente al instante del siguiente evento**. Para eso solo hay que saber *cuándo* ocurre el siguiente evento y *cuál* es. Ambas cosas salen del capítulo 4.

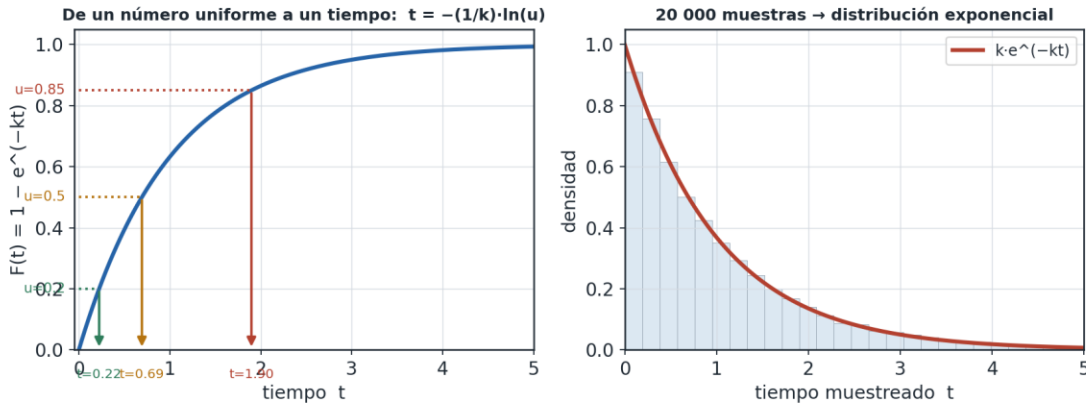
3 Saltar de evento en evento

3.1 Muestrear un tiempo exponencial

Del capítulo 4 sabemos que el tiempo hasta el siguiente evento de una reacción de tasa k es exponencial, con media $1/k$. Para **sortear** uno de esos tiempos en la computadora basta un número aleatorio uniforme $u \in (0, 1)$: invertimos la distribución acumulada $F(t) = 1 - e^{-(kt)}$ = u :

$$t = -(1/k) \cdot \ln(u)$$

(Como $1-u$ también es uniforme, se usa directamente u .) Esto se llama **muestreo por transformada inversa** (Figura 2): un número plano entre 0 y 1 se convierte, a través de la curva acumulada, en un tiempo con la estadística correcta.



3.2 Primera versión: un número por reacción

Para un RNAm con **creación** (tasa k) y **destrucción** (tasa γn), sorteamos un tiempo candidato para cada una: $t_c = -\ln(u_1)/k$ y $t_d = -\ln(u_2)/(\gamma n)$. El que ocurre **primero** es el menor de los dos: avanzamos el tiempo hasta él, ejecutamos ese evento ($n \rightarrow n \pm 1$) y repetimos, volviendo a sortear.

PRIMERA VERSIÓN (MÉTODO DE LA PRIMERA REACCIÓN)

1. Sortear u_1, u_2 ; calcular $t_c = -\ln(u_1)/k$ y $t_d = -\ln(u_2)/(\gamma n)$.
2. El siguiente evento es el del **tiempo menor**; avanzar el reloj hasta ese tiempo.
3. Ejecutar ese evento ($n+1$ si fue creación, $n-1$ si destrucción).
4. Volver a 1.

Es correcto, pero gasta **un número aleatorio por reacción** (aquí, dos por paso) y descarta el mayor. Cuando hay muchas reacciones, sortear un tiempo para cada una y tirar casi todos es un desperdicio. Podemos hacerlo mejor con un hecho muy útil.

4 La suma de exponenciales y el método directo

4.1 El primero de varios relojes exponenciales

Imagina dos “relojes” exponenciales independientes, de tasas k_1 y k_2 . ¿Cuándo suena el **primero**? La probabilidad de que **ninguno** haya sonado hasta t es el producto $e^{-(k_1 t)} \cdot e^{-(k_2 t)} = e^{-(k_1 + k_2)t}$. Es decir, el tiempo al primer evento es de nuevo **exponencial, con tasa la suma $k_1 + k_2$** (Figura 3). Y la probabilidad de que el que sonó fuera el reloj 1 es $k_1/(k_1 + k_2)$: **proporcional a su tasa**.

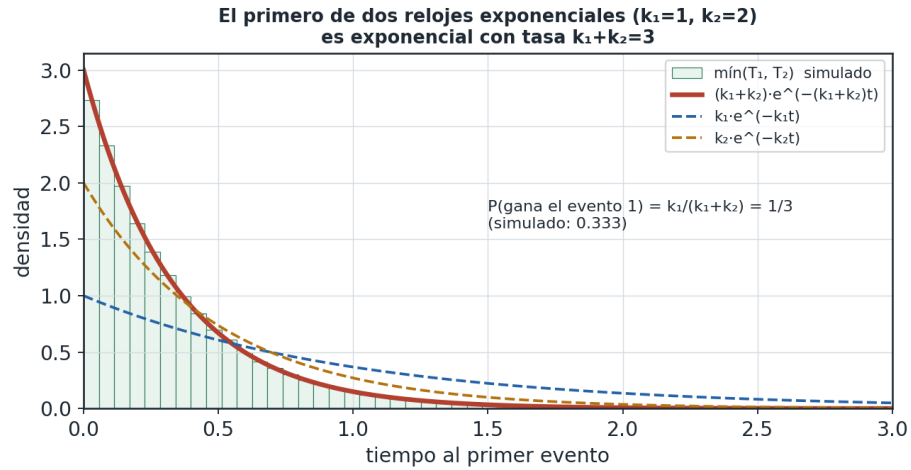


Figura 3. El primero de dos relojes exponenciales ($k_1=1$, $k_2=2$) es exponencial con tasa $k_1+k_2=3$; y cuál de los dos sonó es proporcional a su tasa.

Esto vale para cualquier número de relojes (es, exactamente, el resultado de Doob): el **primer** evento entre muchos ocurre a un tiempo exponencial con tasa $a_0 = \sum a_i$, y **cuál** evento es, es proporcional a cada a_i .

4.2 El método directo

Entonces no hace falta un tiempo por reacción. Con $a_0 = k + \gamma n$ (la suma de todas las tasas), sorteamos **un solo** tiempo al siguiente evento, $\tau = -\ln(u_1)/a_0$, y **un solo** número más u_2 para decidir cuál: creación si $u_2 < k/a_0$, si no, destrucción. Avanzamos el reloj en τ , ejecutamos el evento, recalculamos las tasas y repetimos.

Son **dos números aleatorios por paso**, sin importar el detalle. Esta es la versión que Gillespie llamó el **método directo**.

5 Muchas reacciones y el costo de los números aleatorios

En el caso general hay M reacciones con propensidades $a_1, \dots, a_M$. El tiempo sale igual, $\tau = -\ln(u_1)/a_0$; y **cuál** reacción ocurre se elige repartiendo el intervalo $(0, a_0)$ en tramos tan anchos como cada a_i y viendo en qué tramo cae $u_2 \cdot a_0$ (Figura 4): la primera j tal que $a_1 + \dots + a_j \geq u_2 \cdot a_0$.

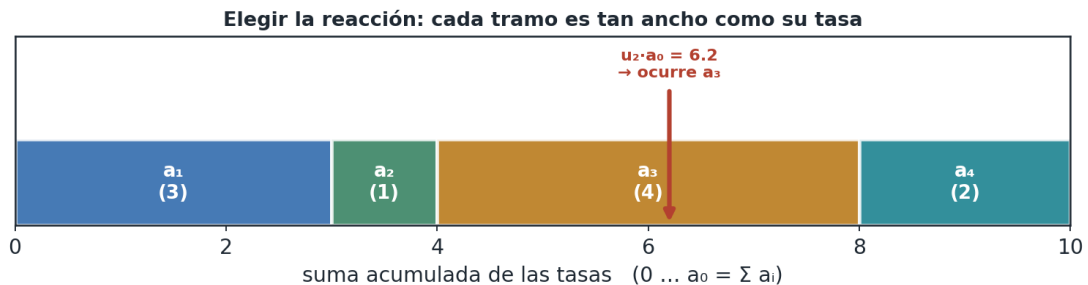


Figura 4. Elegir la reacción: el intervalo $(0, a_0)$ se divide en tramos proporcionales a cada tasa; el dardo $u_2 \cdot a_0$ señala cuál ocurre.

MÉTODO DIRECTO (ALGORITMO DE GILLESPIE, VERSIÓN GENERAL)

- 1. Inicializar el tiempo $t = 0$ y el estado n .
- 2. Calcular todas las propensidades a_i y su suma a_0 .

- 3. Sortear u_1, u_2 ; el siguiente evento ocurre en $\tau = -\ln(u_1)/a_0$; hacer $t \rightarrow t + \tau$.
 - 4. Elegir la reacción j : la primera con $a_1 + \dots + a_j \geq u_2 \cdot a_0$; ejecutarla (actualizar n).
5. Volver a 2 (hasta $t = t_{\max}$).

¿Por qué preferir esto a la primera versión? Porque **generar un número aleatorio es caro** (cuesta muchas operaciones, mucho más que sumar o comparar). El método de la primera reacción gasta M números por paso; el método directo, **solo 2**, sin importar M . A cambio hay que calcular la suma a_0 y buscar el tramo (coste $\sim M$ de aritmética barata). Para redes grandes, el ahorro en números aleatorios es enorme.

Existen refinamientos. El **método de la siguiente reacción** (Gibson y Bruck, 2000) **reutiliza** los números aleatorios con una cola de prioridad y solo recalcula las propensidades que de verdad cambiaron — ideal para circuitos grandes y poco conectados. Todos, en el fondo, son la misma construcción de Doob.

6 El algoritmo en acción

Apliquemos el método directo al RNAm constitutivo (creación k , destrucción γn). Cada corrida es una **escalera** —la historia de una célula—; el promedio de muchas corridas reproduce la curva determinista; y el **histograma en estado estacionario** es la **Poisson** del capítulo 4, con $\lambda = k/\gamma$ (Figura 5).

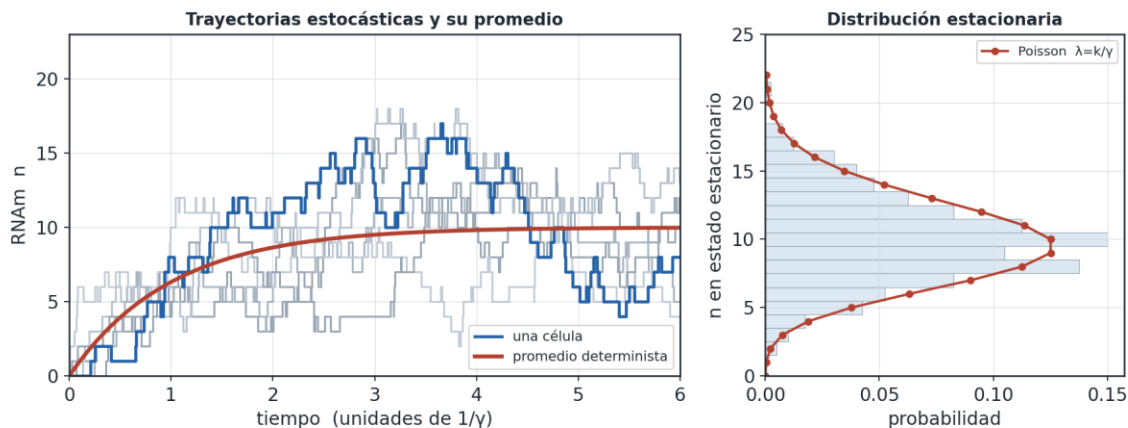


Figura 5. Gillespie aplicado a un RNAm constitutivo. Izquierda: trayectorias estocásticas individuales y el promedio determinista. Derecha: la distribución estacionaria coincide con la Poisson ($\lambda = k/\gamma$) que obtuvimos analíticamente en el capítulo 4.

El algoritmo **reproduce exactamente** los resultados analíticos de los capítulos 4 y 5 — y, a diferencia de ellos, sigue funcionando cuando el circuito es demasiado complejo para resolverlo con lápiz y papel.

7 Métodos híbridos: simular solo donde hace falta

El algoritmo de Gillespie es exacto, pero da **un paso por cada evento**. En la expresión génica eso es un problema: las proteínas son **abundantes** (miles), así que casi todo el cómputo se va en simular nacimientos y muertes de proteínas. Y sin embargo, como aprendimos en el capítulo 5, **ese ruido propio de la proteína es despreciable** justo porque hay muchas. El ruido que

importa en la proteína **viene del RNAm**. Conviene, entonces, simular con detalle solo donde la discreción cuenta —el RNAm, de pocas copias— y tratar a la proteína, abundante, de forma más barata. Dos estrategias híbridas.

7.1 Simular el RNAm, resolver la proteína

Recordemos la descomposición del ruido de la proteína del capítulo 5 ($\eta^2 = \sigma^2/\langle x \rangle^2$, $\phi = \tau_p/\tau_r$):

$$\eta_p^2 = 1/\langle p \rangle + \eta_r^2 \cdot 1/(1+\phi)$$

El primer término es el ruido **propio** de la proteína (Poisson de su creación y destrucción); el segundo es el ruido del **RNAm**, promediado en el tiempo por la vida de la proteína. Cuando $\langle p \rangle$ es grande, $1/\langle p \rangle \approx 0$: prácticamente **todo** el ruido de la proteína viene del RNAm.

La estrategia híbrida es directa: se simula el RNAm $r(t)$ con Gillespie (barato, pocas copias, y ahí la discreción sí cuenta) y se resuelve la proteína de forma **determinista**, impulsada por ese RNAm estocástico:

$$dp/dt = k_p \cdot r(t) - \gamma_p \cdot p$$

Entre eventos del RNAm, r es constante, así que p relaja exponencialmente y se resuelve **analíticamente en cada tramo**. La proteína hereda el ruido (filtrado) del RNAm, pero no su propio ruido de Poisson — que descartamos a propósito porque es insignificante (Figura 6). El resultado captura el ruido biológicamente relevante y es **muchísimo más rápido**, porque no se simula ni un solo evento de proteína.

Híbrido: el ruido de la proteína viene del RNAm, así que solo se simula el RNAm

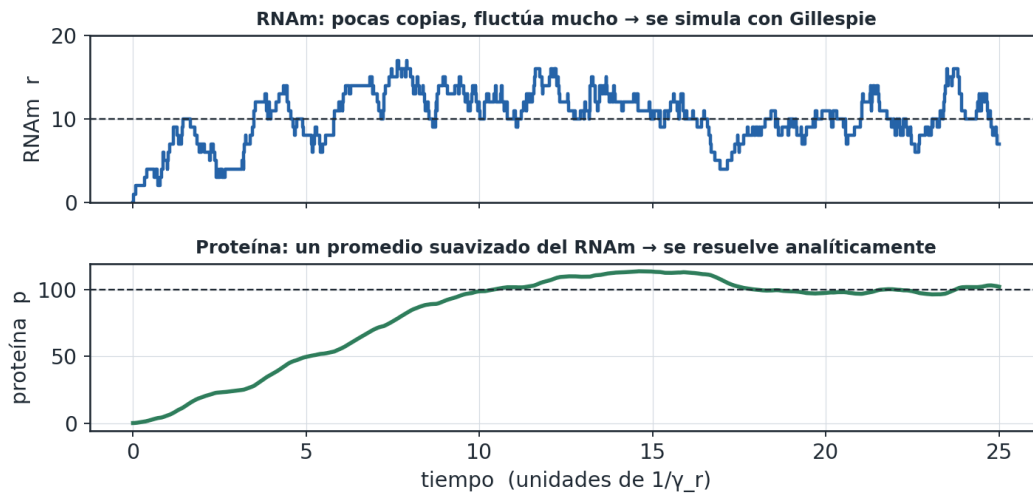


Figura 6. Método híbrido. Se simula el RNAm con Gillespie (arriba) y se integra la proteína de forma determinista impulsada por él (abajo). La proteína es una versión suavizada —filtrada por su vida media— del RNAm: su ruido viene de ahí.

7.2 Bursts sin RNAm: creación a ráfagas, degradación determinista

Cuando el RNAm vive **mucho menos** que la proteína ($\phi = \tau_p/\tau_r \gg 1$, lo habitual), cada RNAm vierte **toda su tanda** de proteínas casi instantáneamente en la escala de la proteína. Podemos entonces **eliminar el RNAm**: cada vez que se produce un RNAm (a tasa k_r) llega una **ráfaga** de proteínas, de tamaño medio $b = k_p/\gamma_r$ (geométrico, porque cada RNAm hace un número aleatorio de proteínas antes de degradarse). Entre ráfagas, la proteína decae.

La simplificación es modelar la **creación como ráfagas discretas** (estocásticas: tasa k_r , tamaño geométrico de media b) y la **degradación como determinista** (decaimiento continuo $-y_p \cdot p$), sin RNAm ni nacimientos individuales de proteína. Este modelo “a ráfagas” reproduce el **ruido dominante** —fijado por el tamaño de ráfaga— y la distribución estacionaria correcta (una **gamma**, o binomial negativa en el conteo discreto) (Figura 7).

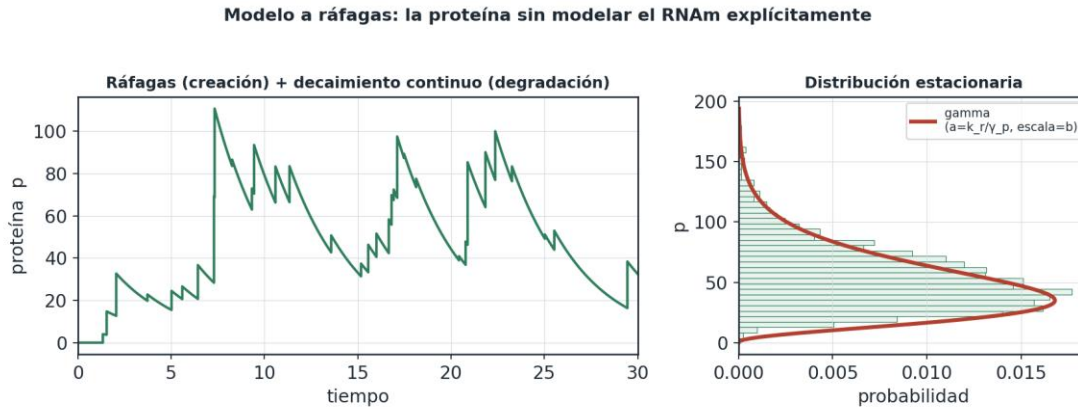


Figura 7. Modelo a ráfagas. La proteína sube a saltos (ráfagas de tamaño medio b , a la tasa del RNAm) y decae de forma continua; su distribución estacionaria es una gamma con forma k_r/y_p y escala b — sin modelar el RNAm.

De aquí sale el resultado célebre de que **el ruido lo fija el tamaño de ráfaga**: $\eta_{p^2} \approx (1+b)/\langle p \rangle$. Y conecta con los experimentos que **observaron directamente** la producción de proteína en ráfagas (Cai, Friedman y Xie, 2006).

8 Cierre: el azar, hecho a mano

Con el algoritmo de Gillespie ya podemos generar, para **cualquier** circuito y de forma **exacta**, las trayectorias de células individuales de las que venimos hablando desde el capítulo 4. Es la herramienta que convierte la ecuación maestra —a menudo irresoluble— en algo que podemos **mirar correr**.

En el próximo capítulo añadiremos más fuentes de ruido biológico —ráfagas de traducción, división celular, ruido extrínseco— que estas simulaciones nos permiten explorar. Y el tema de fondo sigue ahí: el ruido no es un estorbo que haya que promediar, sino una **señal que la célula debe manejar** — y que ahora sabemos simular.

9 Glosario

Algoritmo de Gillespie (SSA): método que genera trayectorias exactas de un proceso de Markov de salto (una realización de la ecuación maestra), saltando de un evento al siguiente.

Propensidad a_i : tasa de la reacción i en el estado actual; la probabilidad de que ocurra en un intervalo dt es $a_i \cdot dt$.

Proceso de Markov: proceso en el que lo que sucede a continuación depende solo del estado actual, no de la historia (capítulo 4).

Muestreo por transformada inversa: generar una variable con distribución dada invirtiendo su acumulada sobre un número uniforme; para la exponencial, $t = -\ln(u)/k$.

Método de la primera reacción: primera versión: se sortea un tiempo candidato por reacción y ocurre el del tiempo menor (M números por paso).

Método directo: versión eficiente: un número da el tiempo (con la tasa total a_0) y otro elige la reacción, proporcional a su tasa (2 números por paso).

Método de la siguiente reacción: refinamiento (Gibson–Bruck) que reutiliza números aleatorios y solo recalcula las propensidades que cambian.

Número (pseudo)aleatorio: valor uniforme en $(0,1)$ que produce la computadora; generarlo es costoso, de ahí que convenga usar pocos por paso.

Método híbrido: simula estocásticamente solo las especies de pocas copias (RNAm) y trata las abundantes (proteína) de forma analítica o a ráfagas, ahorrando cómputo sin perder el ruido relevante.

Ráfaga (burst): tanda de proteínas producida por un mismo RNAm durante su vida; su tamaño medio $b = k_p/\gamma_r$ fija el ruido dominante de la proteína.

10 Herramientas teóricas y lecturas para profundizar

Como en los capítulos anteriores, cada tema pertenece a un campo más amplio. Aquí van el campo y una referencia completa para seguir.

Procesos de Markov y su simulación

El fundamento matemático (tiempos exponenciales, saltos proporcionales a las tasas) y el algoritmo de simulación exacta (§1–§5).

- Doob, J. L. (1953). *Stochastic Processes*. John Wiley & Sons, Nueva York. — La teoría de los procesos de Markov de salto.
- Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25), 2340–2361. — El artículo fundacional del método.
- Gillespie, D. T. (1976). A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of Computational Physics*, 22(4), 403–434.
- Gibson, M. A. y Bruck, J. (2000). Efficient exact stochastic simulation of chemical systems with many species and many channels. *The Journal of Physical Chemistry A*, 104(9), 1876–1889. — El método de la siguiente reacción.

Modelado estocástico en biología de sistemas

Cómo plantear y simular estos modelos en la práctica, con código y ejemplos biológicos.

- Wilkinson, D. J. (2018). *Stochastic Modelling for Systems Biology* (3.^a ed.). Chapman & Hall/CRC, Boca Ratón (EE. UU.). ISBN 978-1-138-54928-9.
- van Kampen, N. G. (1992). *Stochastic Processes in Physics and Chemistry*. North-Holland, Ámsterdam. — La referencia clásica de la ecuación maestra.

Ruido, ráfagas y métodos multiescala

El ruido de la proteína, su origen en el RNAm y la descripción a ráfagas (§7).

- Cai, L., Friedman, N. y Xie, X. S. (2006). Stochastic protein expression in individual cells at the single molecule level. *Nature*, 440(7082), 358–362. — Las ráfagas de proteína, observadas.
- Shahrezaei, V. y Swain, P. S. (2008). Analytical distributions for stochastic gene expression. *PNAS*, 105(45), 17256–17261. — La distribución gamma del modelo a ráfagas.

Próximo capítulo: *más fuentes de ruido — ráfagas, división celular y ruido extrínseco — y sus consecuencias biológicas.*